这几天应朋友邀请，做了一次面试分享，同学们对我介绍的性能优化的方案比较感兴趣，所以整理总结了一下，发出来吧。

1. 性能优化方案
    a. **请求优化**
        i. 请求前
            1. **合理利用缓存**
                a. 本地存储：localStorage，indexedDB等本地存储方案
                b. 浏览器内存缓存Memory Cache
                c. Service Worker和Cache API，拦截请求，查看是否有缓存，提供的缓存可以认为是“永久性”的，关闭浏览器或离开页面之后，下次再访问仍然可以使用
                d. HTTP强缓存disk cache（硬盘缓存），Expires 设置过期时间和 Cache-Control 设置max-age
                e. 协商缓存,先“问一问”服务器资源是否过期，过期了返回200，没过期返回304，Last-Modified&If-Modified-Since， ETag&If-None-Match（一般会文件的 MD5 作为 ETag ）
                f. HTTP2的Push Cache：匹配上就用，就不会额外检查资源是否过期；存活时间很短（Chrome 中为 5min 左右）；只会被使用一次；连接断开就失效，server push缓存那些资源
            2. **DNS预解析**（dns-prefetch），预建立链接(preconnect)，预加载（prefetch，prerender(chrome已弃用)，preload立即获取）
            3. **减少不必要的请求**：浏览器对同源请求的并发限制（chrome是6个），以及TCP的拥塞控制的慢启动特点，考虑请求合并或者按需加载，比如雪碧图，base64 1.3倍，路由懒加载，图片懒加载等，弱网情况下
        ii. 请求中
            1. **避免不必要的重定向**(301永久重定向，302临时重定向)，eg:jd.com -> www.jd.com -> https://www.jd.com
            2. 使用http2，二进制分帧，多路复用，头信息压缩，服务器推送，请求优先级
        iii. 请求后
            1. **渲染优化**：避免强制同步布局，减少重排，重绘
                a. 有一些 JavaScript 操作会导致浏览器需要提前执行布局操作，这种操作就被称为“强制同步布局”，如添加元素，删除元素，修改大小，移动元素位置，获取元素相关信息等
                b. 业务无关脚本使用 async,例如统计脚本、监控脚本、广告脚本等，无外部依赖，不访问DOM，无执行时机限制。（defer）
                c. 使用文档碎片，批量化操作
                动画使用requestAnimationFrame更加流畅

d.
e. 使用composite合成层：设置3D或者css3 transform，z-index，opacity，filter，开启合成层，合成层由GPU合成，重绘时只会重绘当前层，需要注意层爆炸，虽然有浏览器的层压缩机制，但是也有很多无法进行压缩的情况，这就可能出现层爆炸的现象。解决方案：使用3D硬件加速提升动画性能时，最好给元素增加一个z-index属性，人为干扰合成的排序，可以有效减少创建不必要的合成层，提升渲染性能，移动端优化效果尤为明显。

2. **服务端渲染**
   a. 优点：更好的SEO，更快的首屏加载速度
   b. 缺点：增加服务器计算压力
3. **静态资源优化**
   a. 使用CDN加速，dns解析返回负载均衡服务器IP，根据本地IP返回最近的负载最低的缓存服务，如果缓存服务器有资源就返回，没有资源就向源服务器请求，并缓存
   b. **减少包体积：**资源压缩，合并，gzip压缩，tree-shaking，splitChunks提取公共代码(需注意提取粒度)，http2头部压缩
   c. 图片：
      i. 选择合适的图片类型
         1. jpg:有损压缩、体积小、加载快、不支持透明，有锯齿感。使用场景：复杂的、色彩丰富的图片，比如：背景图、轮播图，banner图。
         2. png:无损压缩、质量高、体积大、支持透明，使用场景： 线条感较强、颜色简单且对比强烈的图片，如logo
         3. webp：体积小，支持半透明，压缩比更好，兼容性差，使用时需注意图片存储到本地的场景
         4. base64:减少http请求次数，体积大，增加渲染成本，使用场景：icon
         5. svg:矢量图、体积小、不失真、兼容性好、渲染成本高，使用场景：icon
         6. canvas：定制性强，适合动态渲染，大数据绘制
         7. gif：适合动画，不支持透明，不适合存储彩色图片，（静音的video性能比gif好）
         8. 字体图标，矢量图，不失真，体积小，使用方便，使用场景：icon
      ii. 避免空src图片，因为也会发起请求
      iii. img标签设置alt属性
      iv. 不同环境下加载不同尺寸和像素的图片，使用media媒体查询或者srcset和sizes属性
      v. 较大的图片考虑使用渐进式，先加载缩略图

vi. 雪碧图
vii. 尽量使用css3代替图片
viii. 懒加载和预加载

b. **框架优化（React）**
   i. 组件更新优化：PureComponent & React.memo（hoc返回PureComponent），usememo & usecallback，useRef
   ii. immutable：解决PureComponent浅比较的问题，可以很快速的进行深比较
   iii. reselect优化redux：组件依赖redux中的部分状态没更新，组件就不会更新，react-router-dom中的userSelector用闭包缓存结果实现这个功能
   iv. Error Boundaries
   v. 精确的依赖收集，和更新他的子组件

c. **webpack打包优化**
   i. 压缩，tree-shaking，提取公共代码块 splitChunks，import()懒加载，预加载proload（同步），文件缓存（hash文件名），happypack

d. **用户体验优化**
   i. 骨架屏
   ii. 占位图
   iii. 离线缓存
   iv. webview冷启动，预请求，loading

e. **特殊场景优化**
   i. 大数据渲染
      1. 虚拟列表：使用 position: absolute配合transform: translate3d() 来解决，手动设置展示元素的偏移量。相当于是不断改变这渲染的元素的位置属性和内部的一些节点，不会有频繁的 DOM 创建与销毁
      2. CPU 密集型任务优化：时间切片requestIdleCallback，并行计算Web Worker启用并行线程处理任务
   ii. 防抖&节流
   iii. Passive event listeners，通过addEventListener监听事件时，第三个特性设置{passive:true}，来阻止默认事件，低版本浏览器第三个参数是用来设置是否进行事件捕获的，所以需要考虑兼容性。

f. **代码层面优化**
   i. 异常未捕获，导致页面崩溃
   ii. async await 的不当使用导致并行请求变成了串行请求
   iii. 闭包导致的内存泄漏

    iv. 频繁地 JSON.parse 和 JSON.stringify 处理大对象，复杂的是O(n), fast-json-stringfy, 提前通过 JSON Schema 知道每个字段的类型
    v. CPU 密集型任务导致事件循环 delay 严重
    vi. 正则表达式的灾难性回溯，回溯次数过多，就会导致灾难性回溯，CPU 100%
    vii. 注意程序局部性，比如二维数组的行遍历和列遍历的性能差别
    viii. 降低css选择器的复杂性，使用flex布局
  g. **根据具体页面的性能瓶颈，合理选择优化方案**
2. 性能监控方案
  a. 本地性能测试，lighthouse
  b. 用户侧性能监控，RUM (Real User Monitoring)